

Chapter 7: Direct Methods

Algorithms for Optimization, 2nd Edition (2026)

Kochenderfer & Wheeler

MIT Press

Chapter 7 Overview

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

What are Direct Methods?

Direct methods — also called *zero-order*, *black-box*, *pattern search*, or *derivative-free* methods — rely **solely on the objective function** $f(\mathbf{x})$ to guide the search. They do *not* compute or use the gradient (first derivative) or Hessian (second derivative) of f .

This makes them the method of choice whenever derivatives are unavailable, unreliable, or expensive to obtain — for example, when f is the output of a simulation, an experiment, or a legacy black-box program. Instead of derivatives, direct methods use geometric or heuristic criteria to choose the next point to evaluate

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.1 Cyclic Coordinate Search — Concept I

The simplest direct method is to optimise one variable at a time while holding every other variable fixed. We cycle through each coordinate direction in turn, performing a one-dimensional line search along that direction, and then repeat until convergence. Because each step is aligned with a coordinate axis, the approach is sometimes called *taxicab search* or *coordinate descent*.

Definition: Cyclic Coordinate Search

Given an objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ (where n is the number of variables) and a current point $\mathbf{x}^{(k)}$ (read: “ $\mathbf{x}$ at iteration k ”), the method updates each coordinate one at a time:

$$x_2^{(1)} = \arg \min_{x_2} f(x_1^{(1)}, x_2, x_3^{(1)}, \dots, x_n^{(1)}) \quad (7.1)$$

$$x_3^{(2)} = \arg \min_{x_3} f(x_1^{(1)}, x_2^{(1)}, x_3, \dots, x_n^{(1)}) \quad (7.2)$$

7.1 Cyclic Coordinate Search — Concept II

Notation

In equation (7.1): $x_2^{(1)}$ (“x-sub-2 superscript 1”) means the updated value of the second variable after the first line search. The notation $\arg \min_{x_2}$ (“argmin over x_2 ”) means “the value of x_2 that minimises”. All other variables $x_1^{(1)}, x_3^{(1)}, \dots, x_n^{(1)}$ are held fixed at their current values. The superscript (1) on $x_j^{(1)}$ means the value of variable j after the first full cycle.

Each step is along one of the n **basis vectors** (also called coordinate directions or standard unit vectors):

$$\mathbf{e}^{(i)} \in \mathbb{R}^n, \quad \mathbf{e}_k^{(i)} = \begin{cases} 1 & k = i \\ 0 & \text{otherwise} \end{cases}$$

Notation: Basis Vector

$\mathbf{e}^{(i)}$ (“e superscript i ”) is a vector of length n that is all zeros except for a 1 in position i . For example, in two dimensions: $\mathbf{e}^{(1)} = [1, 0]^\top$ and $\mathbf{e}^{(2)} = [0, 1]^\top$. Line-searching along $\mathbf{e}^{(i)}$ means moving only in the i -th variable.

7.1 Cyclic Coordinate Search — Concept III

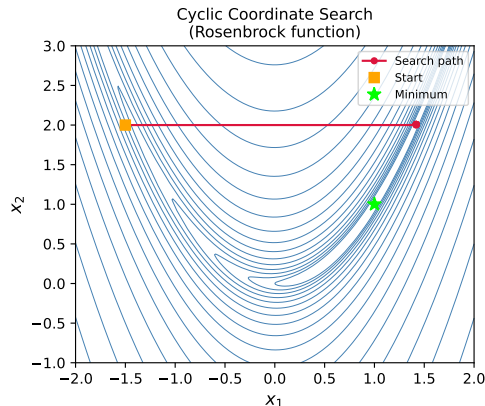
Worked Example (2D)

Let $f(x_1, x_2) = x_1^2 + 2x_2^2$, starting at $\mathbf{x} = [3, 2]^\top$.

Step 1 — optimise over x_1 , holding $x_2 = 2$: We minimise $g_1(x_1) = x_1^2 + 2(2)^2 = x_1^2 + 8$. The minimum is at $x_1 = 0$, so we move to $[0, 2]^\top$.

Step 2 — optimise over x_2 , holding $x_1 = 0$: We minimise $g_2(x_2) = 0 + 2x_2^2$. The minimum is at $x_2 = 0$, so we arrive at $[0, 0]^\top$.

In this example the algorithm converged in one cycle because the function is separable (each variable appears independently). For a function like Rosenbrock where the variables are coupled, many more cycles are needed.



Cyclic coordinate search applied to the Rosenbrock function. The path alternates between the two coordinate directions, zigzagging toward the minimum at (1, 1).

7.1 Cyclic Coordinate Search — Concept IV

Key Insight

Cyclic coordinate search is easy to implement and works well when the objective function is *separable* — meaning each variable contributes independently to f . However, when the level curves of f are tilted diagonally, the method must take many small steps to move diagonally, because each individual step is locked to a coordinate axis. This is the fundamental weakness of the method.

7.1 Cyclic Coordinate Search — Algorithm and Acceleration I

Algorithm 7.1 — Cyclic Coordinate Search

Inputs: the function f , starting point $\mathbf{x}^{(1)}$, and a stopping tolerance ϵ (epsilon, pronounced “eps-ill-on”) — the minimum movement in $\mathbf{x}$ considered significant.

- 1 Set $\Delta = \infty$ (Delta, the change in position), $n = \text{length}(\mathbf{x})$
- 2 **while** $\Delta > \epsilon$:
- 3 Save current position: $\mathbf{x}' \leftarrow \mathbf{x}$
- 4 **for** $i = 1, \dots, n$:
- 5 Set direction $d \leftarrow \mathbf{e}^{(i)}$
- 6 Update: $\mathbf{x} \leftarrow \text{line_search}(f, \mathbf{x}, d)$
- 7 Measure movement: $\Delta \leftarrow \|\mathbf{x} - \mathbf{x}'\|$
- 8 **return** $\mathbf{x}$

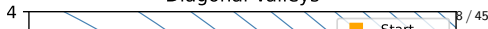
Acceleration Step

After a full cycle that moves from $\mathbf{x}'$ to $\mathbf{x}$, we can add one more line search along the direction of net movement:

$$\mathbf{x}^{(k+1)} \leftarrow \text{line_search}(f, \mathbf{x}, \mathbf{x} - \mathbf{x}')$$

The idea is that if the algorithm consistently moved in a direction over a whole cycle, then taking an additional step in that same direction is likely to be beneficial. This “momentum” step can significantly reduce the number of cycles required to converge on diagonal valleys.

Cyclic Coordinate: Slow on
Diagonal Valleys



7.1 Cyclic Coordinate Search — Python Implementation

Listing 1: Cyclic coordinate search in Python (NumPy/SciPy)

```
1 import numpy as np
2 from scipy.optimize import minimize_scalar
3
4 def cyclic_coordinate_search(f, x0, eps=1e-6, max_cycles=200):
5     """
6     Cyclic coordinate search (coordinate descent with line search).
7     f : objective function  $\mathbb{R}^n \rightarrow \mathbb{R}$ 
8     x0 : initial point (array-like)
9     eps: stopping tolerance on the change in x per cycle
10    """
11    x = np.array(x0, dtype=float)
12    n = len(x)
13    for _ in range(max_cycles):
14        x_prev = x.copy()
15        for i in range(n):
16            # Line search along the i-th coordinate direction.
17            # We fix all other coordinates and minimise over x[i].
18            def fi(t):
19                xc = x.copy()
20                xc[i] = t
21                return f(xc)
22            result = minimize_scalar(fi, bracket=(x[i]-2, x[i]+2),
23                                         method='brent')
24            x[i] = result.x
25    if np.linalg.norm(x - x_prev) < eps:
```

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method**
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.2 Powell's Method — Motivation and Conjugate Directions I

Cyclic coordinate search gets stuck on diagonal valleys because it can only move along axis-aligned directions. Powell's method (Michael J. D. Powell, 1964) fixes this by gradually building up a set of *conjugate* search directions that are better adapted to the shape of the objective function. Unlike the fixed coordinate directions, these new directions can point diagonally, allowing the algorithm to follow curved valleys directly.

Definition: Conjugate Directions

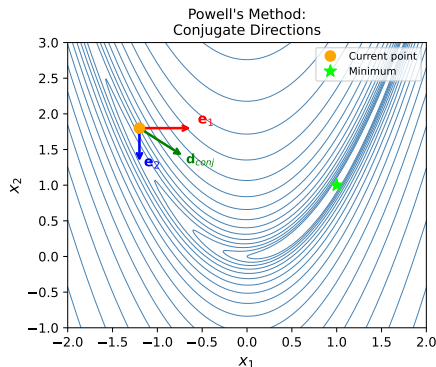
Two directions $\mathbf{u}$ and $\mathbf{v}$ are called **conjugate** with respect to a matrix $\mathbf{Q}$ (the Hessian of f) if:

$$\mathbf{u}^\top \mathbf{Q} \mathbf{v} = 0$$

This is a generalisation of orthogonality.

Notation

Here $\mathbf{u}^\top$ (“ $\mathbf{u}$ transpose”) denotes the row vector version of $\mathbf{u}$, so $\mathbf{u}^\top \mathbf{Q} \mathbf{v}$ is a scalar (a single number) obtained by multiplying a row vector, a matrix, and a column vector



7.2 Powell's Method — Algorithm 1

Algorithm 7.4 — Powell's Method

Inputs: function f , starting point $\mathbf{x}$, tolerance ϵ (epsilon).

- 1 Initialise $U \leftarrow [\mathbf{e}^{(1)}, \dots, \mathbf{e}^{(n)}]$ (the n coordinate basis directions as column vectors)
- 2 Set $\Delta \leftarrow \infty$
- 3 **while** $\Delta > \epsilon$:
 - 4 Save: $\mathbf{x}' \leftarrow \mathbf{x}$
 - 5 **for** $i = 1, \dots, n$: (line search along each direction)
 - 6 $d \leftarrow U[i]$
 - 7 $\mathbf{x} \leftarrow \text{line_search}(f, \mathbf{x}, d)$
 - 8 **for** $i = 1, \dots, n - 1$: (shift directions: drop oldest)
 - 9 $U[i] \leftarrow U[i + 1]$
 - 10 $U[n] \leftarrow \mathbf{x} - \mathbf{x}'$ (add net displacement as new

Reading the Algorithm Step by Step

Step 1 initialises the search directions to the standard coordinate axes — just like cyclic search.

Steps 4–7 perform one full cycle of n line searches, one along each current direction, advancing $\mathbf{x}$ each time.

Steps 8–11 update the direction set. The oldest direction $U[1]$ is discarded and every direction shifts one position. The *net displacement* from the start to the end of the cycle, $\mathbf{x} - \mathbf{x}'$, becomes the new last direction $U[n]$. A final line search is then performed along this new direction starting from $\mathbf{x}'$.

Step 12 measures convergence as the Euclidean distance moved during the cycle.

7.2 Powell's Method — Python Implementation

Listing 2: Powell's method (Python/SciPy)

```
1 import numpy as np
2 from scipy.optimize import minimize_scalar, minimize
3
4 def powell(f, x0, eps=1e-8, max_iter=200):
5     """
6     Powell's method for unconstrained minimisation.
7     f : callable R^n -> R
8     x0 : initial point (1-D array)
9     eps: stopping tolerance
10    """
11    x = np.array(x0, dtype=float)
12    n = len(x)
13    U = np.eye(n)    # start with n coordinate directions
14
15    for _ in range(max_iter):
16        x_prev = x.copy()
17        # Cycle through all current search directions
18        for i in range(n):
19            d = U[i]
20            res = minimize_scalar(lambda t: f(x + t * d))
21            x = x + res.x * d
22        # Shift directions: drop oldest, add net displacement
23        U[:-1] = U[1:]
24        U[-1] = x - x_prev
25        norm = np.linalg.norm(U[-1])
```

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method**
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.3 Hooke-Jeeves Method — Concept I

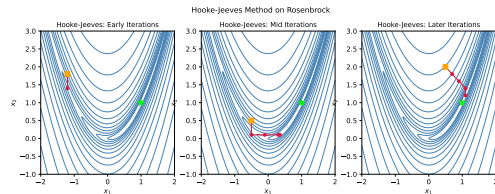
The **Hooke-Jeeves method** (Robert Hooke and T. A. Jeeves, 1961) is a derivative-free method that uses small exploratory steps to probe the neighbourhood of the current point, and then capitalises on any improvement by taking a larger *pattern move* in the same direction. The intuition is simple: if taking a small step in some direction improved the objective, then it is likely worthwhile to continue moving in that direction.

Two-Phase Iteration

Each iteration of the Hooke-Jeeves method has two phases:

Phase 1 — Exploratory Search. Starting from the current *anchor point* $\mathbf{x}^{(\text{base})}$, probe the neighbourhood by trying a small step $\pm\alpha \mathbf{e}^{(i)}$ (“plus or minus alpha times e-superscript-i”) in each coordinate direction i , where α (alpha) is the current step size. Accept whichever probe improves f most. If neither $+\alpha \mathbf{e}^{(i)}$ nor $-\alpha \mathbf{e}^{(i)}$ improves f , keep the current point for dimension i .

Phase 2 — Pattern Move. If the exploratory search



Hooke-Jeeves applied to the Rosenbrock function. Large exploratory steps early on are followed by a step-size reduction when the algorithm approaches the curved valley.

7.3 Hooke-Jeeves Method — Algorithm and Worked Example I

Algorithm 7.5 — Hooke-Jeeves

Inputs: f , starting point $\mathbf{x}$, initial step size α (alpha), tolerance ϵ (epsilon).

- 1 **while** $\alpha > \epsilon$:
- 2 Save anchor: $\mathbf{x}^{(\text{base})} \leftarrow \mathbf{x}$
- 3 **for** $i = 1, \dots, n$: (exploratory)
- 4 Evaluate $f(\mathbf{x} + \alpha \mathbf{e}^{(i)})$ and $f(\mathbf{x} - \alpha \mathbf{e}^{(i)})$
- 5 Keep whichever reduces f ; else keep $\mathbf{x}$
- 6 **if** improvement found (i.e., $\mathbf{x} \neq \mathbf{x}^{(\text{base})}$):
- 7 Pattern step: $\mathbf{x} += \mathbf{x} - \mathbf{x}^{(\text{base})}$
- 8 **else**: (no improvement found)
- 9 $\alpha \leftarrow \alpha/2$ (reduce step size)
- 10 **return** $\mathbf{x}$

Worked Example (2D)

Let $f(x_1, x_2) = x_1^2 + 2x_2^2$, start at $\mathbf{x} = [2, 1]^\top$, initial step $\alpha = 0.5$.

Exploratory phase:

- Try $[2.5, 1]$: $f = 6.25 + 2 = 8.25$ (worse)
- Try $[1.5, 1]$: $f = 2.25 + 2 = 4.25$ (better ✓)
- Try $[1.5, 1.5]$: $f = 2.25 + 4.5 = 6.75$ (worse)
- Try $[1.5, 0.5]$: $f = 2.25 + 0.5 = 2.75$ (better ✓)

After exploration: $\mathbf{x} = [1.5, 0.5]^\top$.

Pattern move: Displacement
 $= [1.5, 0.5] - [2, 1] = [-0.5, -0.5]$. New point:

7.3 Hooke-Jeeves Method — Algorithm and Worked Example II

Convergence Properties

The Hooke-Jeeves method is provably convergent for continuous functions: as the step size α shrinks to zero, the sequence of iterates must accumulate at a stationary point. However, like all local methods it can converge to a local minimum rather than the global minimum. The rate of convergence is linear (i.e., the error reduces by a constant factor each iteration), which is slower than methods that use derivative information.

7.3 Hooke-Jeeves — Python Implementation

Listing 3: Hooke-Jeeves method in Python

```
1 import numpy as np
2
3 def hooke_jeeves(f, x0, alpha=1.0, eps=1e-6):
4     """
5     Hooke-Jeeves (pattern search) method.
6     f      : callable  $R^n \rightarrow R$ 
7     x0     : initial point
8     alpha  : initial step size (alpha, controls exploration radius)
9     eps    : stopping tolerance (stop when alpha < eps)
10    """
11    x = np.array(x0, dtype=float)
12    n = len(x)
13    while alpha > eps:
14        x_base = x.copy()      # save the anchor point
15        x_trial = x.copy()    # start exploration from here
16        improved = False
17        for i in range(n):
18            for sign in (1, -1):
19                x_new = x_trial.copy()
20                x_new[i] += sign * alpha
21                if f(x_new) < f(x_trial):
22                    x_trial = x_new
23                    improved = True
24                    break        # accept first improvement
25    if improved:
```

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search**
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.4 Generalized Pattern Search — Overview I

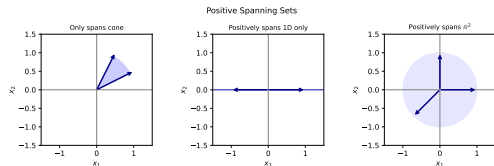
The Hooke-Jeeves method uses the $\pm \mathbf{e}^{(i)}$ coordinate directions for its exploratory steps. **Generalized Pattern Search** (GPS) relaxes this restriction and allows *any* set of directions $\mathcal{D}$, provided that set satisfies a key geometric condition: it must be a *positive spanning set*. This generalisation allows the search to explore diagonal directions from the start, and also enables the method to handle constrained problems where some coordinate directions would leave the feasible region.

Definition: Positive Spanning Set

A set of vectors $\mathcal{D} = \{\mathbf{d}^{(1)}, \dots, \mathbf{d}^{(m)}\}$ is a **positive spanning set** for $\mathbb{R}^n$ if every vector in $\mathbb{R}^n$ can be written as a **non-negative** linear combination of vectors in $\mathcal{D}$. Formally:

$$\forall \mathbf{v} \in \mathbb{R}^n, \exists \lambda^{(i)} \geq 0 : \mathbf{v} = \sum_{i=1}^m \lambda^{(i)} \mathbf{d}^{(i)}$$

At least $m \geq n + 1$ directions are required.



Three sets of directions in $\mathbb{R}^2$. Left: only spans a cone (not all of $\mathbb{R}^2$). Centre: only spans one dimension. Right: positively spans all of $\mathbb{R}^2$ — any direction can be expressed as a non-negative combination of the shown vectors.

7.4 Generalized Pattern Search — Algorithm and Mesh I

Algorithm 7.6 — Generalized Pattern Search

Inputs: f , starting point $\mathbf{x}$, initial step α (alpha), direction set $\mathcal{D}$, tolerance ϵ .

- 1 **while** $\alpha > \epsilon$:
- 2 **for** each direction $\mathbf{d} \in \mathcal{D}$:
- 3 **if** $f(\mathbf{x} + \alpha\mathbf{d}) < f(\mathbf{x})$:
- 4 Accept: $\mathbf{x} \leftarrow \mathbf{x} + \alpha\mathbf{d}$; **break** (stop iterating $\mathcal{D}$)
- 5 **if** no direction gave improvement:
- 6 Reduce: $\alpha \leftarrow \alpha/2$
- 7 **return** $\mathbf{x}$

The Mesh Structure

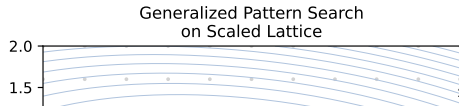
A key theoretical property of GPS is that all evaluated points lie on a scaled *lattice* (a regularly-spaced grid):

$$\mathbf{x}^{(k)} = \mathbf{x}^{(1)} + \alpha^{(k)} \mathbf{D} \mathbf{z}, \quad \mathbf{z} \in \mathbb{Z}_{\geq 0}^m$$

where $\mathbf{D}$ is the matrix whose columns are the direction vectors, and $\mathbf{z}$ (z) is a vector of non-negative integers recording how many steps have been taken in each direction. The lattice is scaled by $\alpha^{(k)}$ (alpha at iteration k) and need not be axis-aligned.

Reading the Algorithm

At each step, the algorithm tries moving in every direction in $\mathcal{D}$ (scaled by the current step size α) and accepts the *first* direction that improves f . If none



7.4 Generalized Pattern Search — Python Implementation

Listing 4: Generalized pattern search in Python

```
1 import numpy as np
2
3 def generalized_pattern_search(f, x0, alpha=1.0, eps=1e-6,
4                               directions=None):
5     """
6     Generalized Pattern Search with a user-supplied positive spanning set.
7     f          : callable  $R^n \rightarrow R$ 
8     x0         : initial point
9     alpha      : initial step size
10    eps        : stopping tolerance
11    directions : array of shape (m, n); defaults to  $\pm e_i$  (2n directions)
12    """
13    x = np.array(x0, dtype=float)
14    n = len(x)
15    if directions is None:
16        # Default: coordinate directions + their negatives (2n vectors)
17        # This is a positive spanning set for  $R^n$ 
18        directions = np.vstack([np.eye(n), -np.eye(n)])
19
20    while alpha > eps:
21        improved = False
22        for d in directions:
23            x_trial = x + alpha * d
24            if f(x_trial) < f(x):
25                x = x_trial
```

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method**
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.5 Nelder-Mead — Simplex Concept and Setup I

The **Nelder-Mead simplex method** (John Nelder and Roger Mead, 1965) maintains a geometric shape called a *simplex* and repeatedly deforms it to move toward and shrink around the minimum. Rather than searching along fixed directions, the simplex is a living shape that adapts to the landscape of f . It is arguably the most widely used derivative-free method in practice, and is the default method in SciPy's `minimize` function when no gradient is provided.

What is a Simplex?

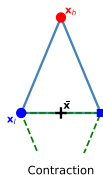
A **simplex** in $\mathbb{R}^n$ is the simplest possible n -dimensional shape: it has exactly $n + 1$ vertices.

- In $n = 1$ dimension: a line segment (2 endpoints)
- In $n = 2$ dimensions: a triangle (3 vertices)
- In $n = 3$ dimensions: a tetrahedron (4 vertices)

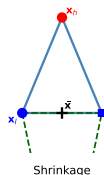
The idea is to evaluate f at all $n + 1$ vertices and then move the *worst* vertex (the one with the highest function value) to a better location using geometric operations.

Nelder-Mead Simplex Operations

Reflection



Expansion



7.5 Nelder-Mead — Four Operations I

Every iteration of Nelder-Mead chooses exactly one of four geometric operations to apply to the worst vertex. The choice depends on how well the candidate new point compares to the existing vertices.

Operation 1: Reflection (α , “alpha” > 0)

The most basic operation: reflect the worst vertex $\mathbf{x}_h$ over the centroid $\bar{\mathbf{x}}$:

$$\mathbf{x}_r = \bar{\mathbf{x}} + \alpha(\bar{\mathbf{x}} - \mathbf{x}_h)$$

This moves $\mathbf{x}_h$ to the *opposite side* of the centroid.

Typically $\alpha = 1$, giving $\mathbf{x}_r = 2\bar{\mathbf{x}} - \mathbf{x}_h$.

The idea: if the minimum is in the direction away from the worst vertex, reflecting moves us toward it.

Operation 2: Expansion (β , “beta” $> \max(1, \alpha)$)

If the reflected point $\mathbf{x}_r$ is the best point seen so far

Operation 4: Shrinkage

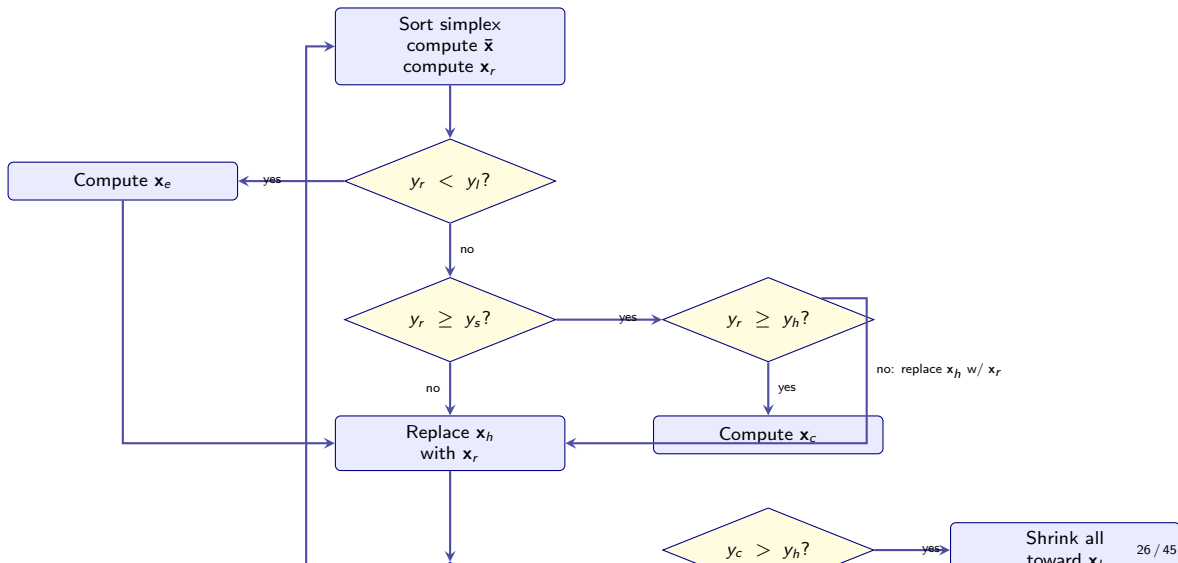
If none of the above yields improvement, shrink the *entire* simplex toward the best point $\mathbf{x}_l$:

$$\mathbf{x}^{(i)} \leftarrow \frac{\mathbf{x}^{(i)} + \mathbf{x}_l}{2}, \quad i \neq l$$

This moves every vertex (except the best) halfway toward the current best. Shrinkage is expensive because it requires re-evaluating f at n new points. It is used as a last resort when the simplex is poorly shaped.

Convergence Criterion

7.5 Nelder-Mead — Decision Flowchart



7.5 Nelder-Mead — Python Implementation

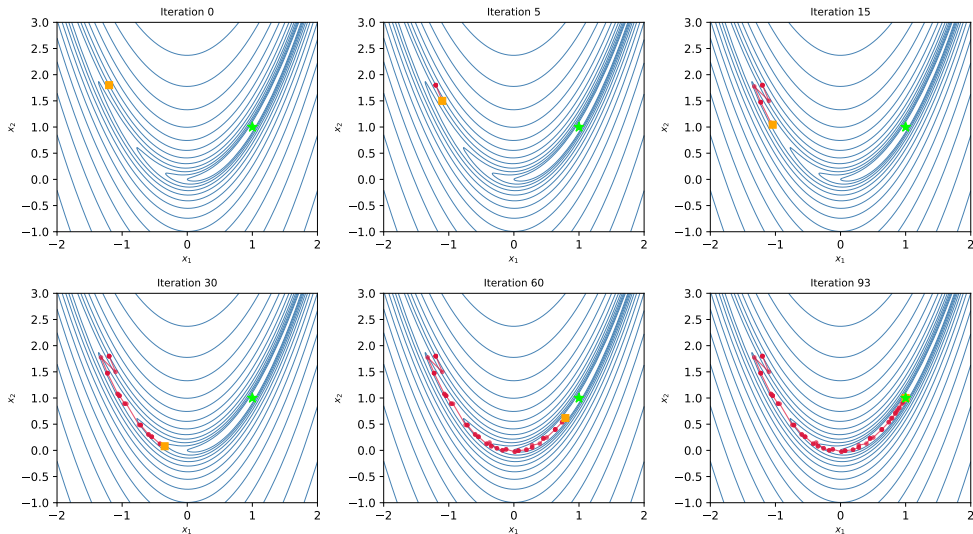
Listing 5: Nelder-Mead simplex method (pure Python + SciPy)

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def nelder_mead(f, simplex, eps=1e-6, alpha=1.0, beta=2.0, gamma=0.5):
5     """
6     Nelder-Mead simplex method.
7     f      : callable  $R^n \rightarrow R$ 
8     simplex : list/array of (n+1) initial vertices, shape (n+1, n)
9     alpha   : reflection coefficient (> 0, typically 1.0)
10    beta     : expansion coefficient (> 1, typically 2.0)
11    gamma    : contraction coefficient (in (0,1), typically 0.5)
12    """
13    S = [np.array(s, dtype=float) for s in simplex]
14    while True:
15        y = np.array([f(s) for s in S])
16        p = np.argsort(y)
17        S = [S[i] for i in p]; y = y[p]
18        if np.std(y) < eps:      # convergence check
19            break
20        xl, yl = S[0], y[0]      # best vertex
21        xh, yh = S[-1], y[-1]    # worst vertex
22        xs, ys = S[-2], y[-2]    # second-worst vertex
23        xm = np.mean(S[:-1], axis=0) # centroid (excluding worst)
24        xr = xm + alpha * (xm - xh)  # reflection
25        yr = f(xr)
```

7.5 Nelder-Mead — Iterations Visualised I

7.5 Nelder-Mead — Iterations Visualised II

Nelder-Mead on Rosenbrock (12 iterations shown)



The figure shows Nelder-Mead applied to the Rosenbrock function $f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$, which

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)**
- 7 Summary & Exercises

7.6 DIRECT — Overview and Lipschitz Condition I

The methods we have studied so far — cyclic search, Powell, Hooke-Jeeves, GPS, Nelder-Mead — are all *local* methods. They converge to a minimum near the starting point but may miss the global minimum. **DIRECT** (Divided RECTangles, Jones et al. 1993) is designed for *global* optimisation: it provides a principled way to search the entire bounded domain $[a, b]$ and is guaranteed to converge to the global minimum given enough iterations.

The Setting: Bounded, Lipschitz-Continuous f

DIRECT applies to problems of the form:

$$\min_{\mathbf{x} \in [a, b]} f(\mathbf{x})$$

where $a, b \in \mathbb{R}^n$ are known lower and upper bounds on each variable, and f is *Lipschitz-continuous* — meaning it cannot change too rapidly.

The Lipschitz Condition

The function f satisfies a Lipschitz condition with

Lower Bound from the Lipschitz Condition

Given that f is evaluated at a center point $\mathbf{c}$, and that any point $\mathbf{x}$ in an interval around $\mathbf{c}$ is at most distance r (the radius) from $\mathbf{c}$, the Lipschitz condition provides a lower bound:

$$f(\mathbf{x}) \geq f(\mathbf{c}) - \ell r \quad \text{for all } \mathbf{x} \text{ in the interval}$$

This says: the smallest f can be anywhere in the interval is $f(\mathbf{c}) - \ell r$. We do not know ℓ , but DIRECT avoids this by considering *all*

7.6 DIRECT — Potentially Optimal Intervals I

The key challenge in global optimisation is deciding *which* region to explore next. Exploring every region equally is wasteful; ignoring a region risks missing the global minimum. DIRECT uses the concept of *potentially optimal intervals* to balance exploration (sampling new regions) with exploitation (refining promising regions).

Definition: Potentially Optimal Interval

An interval I with center $\mathbf{c}$ and radius r (half-diagonal) is **potentially optimal** if there exists some Lipschitz constant $\ell \geq 0$ such that both conditions hold:

- 1 $f(\mathbf{c}) - \ell r \leq f(\mathbf{c}') - \ell r'$ for every other interval I' with center $\mathbf{c}'$ and radius r' . (This interval's lower bound is as good as any other's.)
- 2 $f(\mathbf{c}) - \ell r \leq f_{\min} - \epsilon_{\text{DIRECT}} |f_{\min}|$ where $f_{\min}$ is the best value found so far. (The potential improvement is worth exploring.)

Splitting Rule

When an interval is selected as potentially optimal, it is split along its *longest* dimension into **three equal parts**. The two new center points are evaluated, and the three sub-intervals replace the original:

$$r_{\text{new}} = \frac{r_{\text{old}}}{3}$$

Splitting along the longest dimension first ensures that all dimensions are eventually refined uniformly. The sub-interval with the smallest function value at its center is split

Geometric Interpretation

7.6 DIRECT — Algorithm I

Algorithm 7.8 — DIRECT

Inputs: f , lower bounds $\mathbf{a}$, upper bounds $\mathbf{b}$, maximum iterations $k_{\max}$, minimum radius $r_{\min}$.

- ➊ **Normalise** the domain: define a helper function $g(\mathbf{x}') = f(\mathbf{x}' \cdot (\mathbf{b} - \mathbf{a}) + \mathbf{a})$ that maps the unit hypercube $[0, 1]^n$ back to $[\mathbf{a}, \mathbf{b}]$.
- ➋ Initialise a single interval: center $\mathbf{c} = [0.5, \dots, 0.5]^\top$, radius $r = \sqrt{n} \cdot 0.5$, zero splits in all dimensions.
- ➌ Evaluate $g(\mathbf{c})$.
- ➍ **for** $k = 1, \dots, k_{\max}$:
- ➎ Identify all *potentially optimal* intervals from the lower convex hull: call this set $\mathcal{S}_{\text{split}}$.
- ➏ Remove those intervals from the collection $\mathcal{S}$.
- ➐ For each interval in $\mathcal{S}_{\text{split}}$: split it into three, evaluate the two new centers, add all three sub-intervals back to $\mathcal{S}$.

The DirectRectangle Data Structure

Each interval is stored as a record with four fields:

- $\mathbf{c}$: the center point in the normalised unit hypercube $[0, 1]^n$
- $y = g(\mathbf{c})$: the function value at the center
- $\mathbf{d} \in \mathbb{Z}_{\geq 0}^n$ ($\mathbf{d}$, integer vector): the number of splits per dimension so far
- r : the half-diagonal radius (used for convex hull selection)

Implementation Note

An efficient implementation groups intervals by their radius value r and, within each group, sorts by $f(\mathbf{c})$. This allows

7.6 DIRECT — Python Implementation

Listing 6: DIRECT algorithm in Python

```
1 import numpy as np
2 from dataclasses import dataclass, field
3 from typing import List
4
5 @dataclass
6 class Rect:
7     c: np.ndarray          # center in unit hypercube  $[0,1]^n$ 
8     y: float               #  $f(c)$ : function value at center
9     d: np.ndarray          # number of splits per dimension (integer)
10    r: float                # half-diagonal radius
11
12 def direct(f, a, b, k_max=20, r_min=1e-4):
13     """DIRECT algorithm for global optimisation over  $[a, b]$ ."""
14     a, b = np.asarray(a, float), np.asarray(b, float)
15     n = len(a)
16     # Map unit hypercube back to  $[a, b]$  for evaluating  $f$ 
17     g = lambda xp: f(xp * (b - a) + a)
18     c = np.full(n, 0.5)
19     rects = [Rect(c.copy(), g(c), np.zeros(n, int),
20                  np.sqrt(n) * 0.5)]
21     c_best = c.copy()
22     for _ in range(k_max):
23         # Find potentially optimal intervals (lower convex hull)
24         to_split = _potentially_optimal(rects, r_min)
25         new_rects = []
```

7.6 DIRECT — Worked Example I

Example 7.1 — Flower Function on $[-1, 3] \times [-2, 1]$

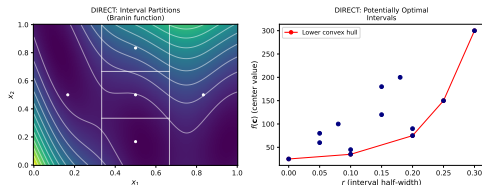
We want to minimise the *flower function* over the rectangle $x_1 \in [-1, 3]$, $x_2 \in [-2, 1]$.

Normalisation. DIRECT first maps the domain to the unit square $[0, 1]^2$ via: $x_1 = 4x'_1 - 1$, $x_2 = 3x'_2 - 2$.

Iteration 0. Sample at the center of the unit square, $[0.5, 0.5]^\top$, which maps to $[1, -0.5]^\top$ in original coordinates. Evaluate $g([0.5, 0.5]) = 0.158$.

First Split. The single initial interval is trivially potentially optimal and is split along dimension 1 (the x'_1 axis). This creates five intervals (the original center plus two new centers for each split):

Int.	Center	Width	r	$f(\mathbf{c})$
1	$[1/6, 3/6]$	$[1/3, 1]$	0.527	0.500
2	$[5/6, 3/6]$	$[1/3, 1]$	0.527	1.231



After several DIRECT iterations on the Branin function.

Left: spatial partitions show finer resolution near the two minima. Right: the $(r, f(\mathbf{c}))$ scatter with the lower convex hull in red, identifying the intervals to be split next.

Reading the Table

Each row corresponds to one interval (a rectangle in the normalised domain). The column “Width” gives the side lengths of the rectangle; “ r ” is the half-diagonal

Outline

- 1 Cyclic Coordinate Search
- 2 Powell's Method
- 3 Hooke-Jeeves Method
- 4 Generalized Pattern Search
- 5 Nelder-Mead Simplex Method
- 6 Divided Rectangles (DIRECT)
- 7 Summary & Exercises

7.7 Summary I

The Role of Direct Methods

Direct methods occupy an important niche in optimisation: they require only the ability to evaluate $f(\mathbf{x})$, making them applicable to simulation-based, experimental, or legacy black-box problems where derivatives are simply unavailable. The trade-off is that they typically require more function evaluations than gradient-based methods, and most of them offer only local convergence guarantees.

Cyclic Coordinate Search

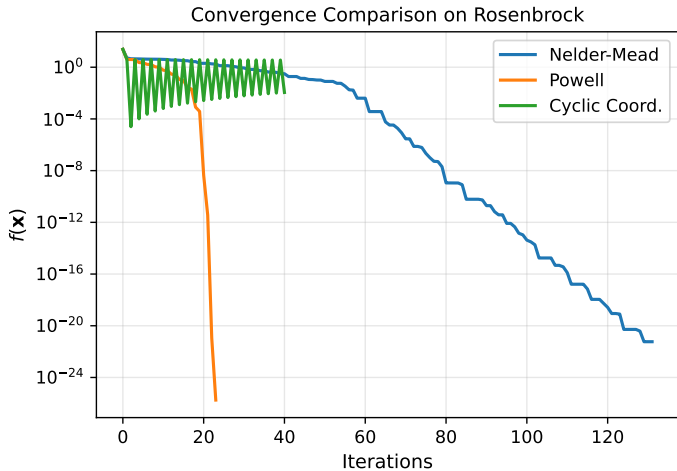
Optimises one variable at a time via line search, cycling through all n coordinate directions repeatedly. It is the simplest direct method and works well for *separable* functions where the variables affect f independently. It is slow on diagonally-oriented valleys because it cannot move diagonally. An acceleration step (adding the net-displacement direction after each cycle) can significantly improve speed.

Generalized Pattern Search

Generalises Hooke-Jeeves to allow any positive spanning set $\mathcal{D}$ of directions, not just the coordinate axes. All evaluated points lie on a scaled lattice. Under mild continuity conditions, convergence to a stationary point is guaranteed — stronger than Hooke-Jeeves. Particularly useful when coordinate directions are inefficient or infeasible.

Nelder-Mead Simplex

7.7 Summary II



7.7 Summary III

Convergence comparison on the Rosenbrock function. Nelder-Mead and Powell's method converge significantly faster (in terms of function evaluations) than cyclic coordinate search. This illustrates the benefit of adaptively building better search directions.

Method Comparison Table

Method	Global?	Evals/iter	Memory	Best for
Cyclic coord.	No	n	$O(n)$	Separable f
Powell	No	$n \cdot (\text{line search})$	$O(n^2)$	Smooth f
Hooke-Jeeves	No	$2n$	$O(n)$	General f
GPS	No	$ \mathcal{D} $	$O(n \mathcal{D})$	Constrained lattice
Nelder-Mead	No	$1-n+1$	$O(n^2)$	Low-dim, smooth f
DIRECT	Yes	$2n$ per split	$O(3^k)$	Bounded, Lipschitz f

7.7 Summary IV

Key Takeaway

There is no single best direct method. The choice depends on the problem: for smooth, low-dimensional problems Nelder-Mead is a reliable first choice; for problems with structure (separability or known geometry) cyclic search or GPS may be more efficient; for global search over a bounded domain, DIRECT is the principled choice. All direct methods share the fundamental trade-off that more function evaluations yield better accuracy, but with diminishing returns as the algorithm hones in on the minimum.

7.8 Exercises I

The following exercises are designed to deepen understanding of the methods in this chapter. Some require pencil-and-paper analysis; others suggest computational experiments.

Exercise 7.1 — Hooke-Jeeves Failure Case

Design an objective function f and a starting point $\mathbf{x}^{(0)}$ such that the Hooke-Jeeves method fails to reduce f at the very first step (i.e., the exploratory phase finds no improvement in any coordinate direction with step size α). What does this imply about the algorithm's behaviour? Does the algorithm eventually recover and converge, or does it stall permanently?

Hint: Consider a function whose minimum lies in a region that is never reached by coordinate-aligned steps of size α from the starting point. Also consider what happens to α as the algorithm progresses.

7.8 Exercises II

Exercise 7.2 — Why Cyclic Search Improves Each Cycle

Explain in your own words why cyclic coordinate search is guaranteed to find an improvement (or confirm a minimum) after each full cycle through all n coordinates, provided the current point is not a local minimum. Then identify the condition on f that breaks this guarantee, giving a specific example.

Hint: Think about what “not a local minimum” means for a function that is differentiable. Then consider a function with a flat plateau or a non-differentiable kink.

Exercise 7.3 — GPS Beats Cyclic Search

Give a concrete two-dimensional objective function $f(x_1, x_2)$ for which cyclic coordinate search with the standard $\pm \mathbf{e}^{(i)}$ directions will stall at a non-optimal point (i.e., every step in a coordinate direction increases f , even though a diagonal step would decrease it). Then describe how GPS with a richer direction set $\mathcal{D}$ that includes diagonal directions would succeed in finding a better point from the same starting location.

7.8 Exercises III

Exercise 7.4 — DIRECT vs. Nelder-Mead: Theoretical Comparison

Explain the fundamental structural difference between the DIRECT algorithm and the Nelder-Mead simplex method. Your answer should address: (a) what each method maintains in memory, (b) how each method decides where to evaluate f next, and (c) what convergence guarantee (if any) each method provides.

Solution sketch: DIRECT maintains a collection of non-overlapping rectangles that tile the entire search domain, whereas Nelder-Mead maintains a single simplex of $n + 1$ points that moves freely through the space. DIRECT's selection of intervals via the lower convex hull gives a global coverage guarantee; Nelder-Mead has no such guarantee. For a 2D problem with $n = 2$, each split of DIRECT evaluates $2n = 4$ new points; after $k = 5$ iterations the total number of rectangles grows at most as $O(3^5) = 243$, though typically far fewer are split each iteration.

7.8 Exercises IV

Exercise 7.5 — Computational Experiment (Multimodal)

Find the global minimum of the multimodal function:

$$f(x_1, x_2) = \sin\left(\frac{1}{2}x_1^2 - \frac{1}{4}x_2^2 + 3\right) \cos(2x_1 + 1 - e^{x_2})$$

over $x_1, x_2 \in [-2, 2]$ using the two approaches below. Compare the solutions found and discuss which method is more reliable for this multimodal (many-local-minima) function.

(a) Nelder-Mead: Run the algorithm from three different starting simplices (e.g., centred at $[-1.5, -1.5]$, $[0, 0]$, and $[1.5, 1.5]$). Use: `scipy.optimize.minimize(f, x0, method='Nelder-Mead')`.

(b) DIRECT: Run the global search directly using: `scipy.optimize.direct(f, bounds=[(-2, 2), (-2, 2)])`.

Hint: Nelder-Mead is sensitive to the starting simplex on this function and may find different local minima from different starting points. DIRECT should reliably find the global minimum regardless of initialisation.

References

-  M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026.
-  R. Hooke and T. A. Jeeves, “Direct Search Solution of Numerical and Statistical Problems,” *Journal of the ACM*, vol. 8, no. 2, pp. 212–229, 1961.
-  J. A. Nelder and R. Mead, “A Simplex Method for Function Minimization,” *The Computer Journal*, vol. 7, no. 4, pp. 308–313, 1965.
-  D. R. Jones, C. D. Perttunen, and B. E. Stuckman, “Lipschitzian Optimization Without the Lipschitz Constant,” *Journal of Optimization Theory and Applications*, vol. 79, no. 1, pp. 157–181, 1993.
-  R. G. Regis, “On the Properties of Positive Spanning Sets and Positive Bases,” *Optimization and Engineering*, vol. 17, no. 1, pp. 229–262, 2016.